Integration Methods in CppODE

Simon Beyer

2026-05-06

Contents

1 Overview

CppODE generates and compiles per-model C++ code for three related problems through three entry points that share a common R interface: `CppODE()`, `CVODE()`, and `funCpp()`. Table 1 summarises their scope.

	`CppODE()`	`CVODE()`	`funCpp()`
Problem class	ODE	ODE	algebraic $y = g(x, p)$
Methods	`bdf, adams, rb4, tsit5`	`bdf, adams`	
First-order sensitivities	in-tree dual-number AD	SUNDIALS analytic RHS	in-tree dual-number AD
Second-order sensitivities	yes (nested duals)	no	yes (symbolic)
External dependencies	LAPACK; KLU optional	SUNDIALS ≥ 6.0; KLU opt.	none
Codegen	SymPy + `R CMD SHLIB`	SymPy + `R CMD SHLIB`	SymPy + `R CMD SHLIB`

Table 1: The three CppODE entry points and what they cover. The codegen pipeline (parse the user's expressions through SymPy, emit C++, compile via `R CMD SHLIB`, load via `dyn.load`) is shared, as is the `solveODE()` dispatcher used by both integrator backends.

The integrator entry points `CppODE()` and `CVODE()` expose the same R-side interface: the user supplies named character vectors of right-hand sides together with optional events, forcings, and parameter vectors, and the returned model handle is passed to `solveODE()`. The two backends differ only in the numerical core. `CppODE()` emits a self-contained C++ source that includes the headers under `inst/include/cppode/`; the resulting binary depends on LAPACK and, optionally, KLU. `CVODE()` emits a thin glue layer that links against the system-installed SUNDIALS CVODE/CVODES library and inherits its numerical behaviour.

Section 2 describes the in-tree integrator `CppODE()` in full, including its four time-stepping methods, its forward-mode automatic-differentiation framework for parameter sensitivities, and the transport of sensitivities across events. Section 3 describes the SUNDIALS-backed `CVODE()` integrator. Section 4 describes the algebraic-function compiler `funCpp()`, which compiles a system of expressions $y = g(x, p)$ together with optional first- and second-order derivatives, with no time integration. The principal use cases of `funCpp()` are observation maps that link integrator output to data in likelihood-based inference and reparametrisation Jacobians $\partial p/\partial\theta$ that seed sensitivity propagation through the integrators.

2 `CppODE()`

2.1 Initial value problem and four methods

This section describes the in-tree integrator of `CppODE()` in detail: the four time-stepping methods, the adaptive step-size control they share, dense Hermite output, sparse and dense LU factorisation, time- and root-triggered events, and forward-mode sensitivity propagation through dual numbers. The methods differ only in how the individual time step is taken. The Butcher tableaus and numerical coefficients of each method are reproduced in Appendices A–D.

The initial-value problem solved by `CppODE()` is

$$\dot{x} = f(x,\, p_{\text{dyn}},\, t), \qquad x(t_0) = p_{\text{init}}, \tag{1}$$

with state $x \in \mathbb{R}^{n_x}$ and parameter vector $p = (p_{\text{init}}, p_{\text{dyn}})$ collecting the initial conditions and the dynamic parameters. The right-hand side f may depend explicitly on time, in which case the system is non-autonomous; an explicit time argument is also necessary for the forcing-function machinery used to inject time-dependent inputs. The Jacobian of f with respect to the state is denoted $J = \partial f/\partial x$ throughout.

2.2 Stiffness and the choice of method

The choice between the four methods is governed primarily by whether the problem is *stiff*. A problem is stiff on a given integration interval when its solution contains components evolving on time scales much shorter

than the interval length. An explicit integration method applied to such a problem is forced to take step sizes set by the fastest component rather than by the accuracy requirement, leading to step counts that are uneconomic in practice. Implicit methods avoid this constraint by absorbing the fast components into a linear solve at each step.

The standard tool for analysing the stability of a method is the linear scalar test equation $\dot{y} = \lambda y$ with $\lambda \in \mathbb{C}^-$, whose exact solution decays. Substituting this equation into a one-step method produces a single scalar amplification factor $R(z)$ acting on y per step, where $z = h\lambda$ combines the step size and the eigenvalue. The method is *A-stable* if $|R(z)| \le 1$ on the entire left half-plane, so that the numerical solution decays whenever the exact one does, regardless of step size; *A(α)-stable* if the same bound holds on the wedge $\{z \in \mathbb{C} : |\arg(-z)| \le \alpha\}$ for some opening angle $\alpha < 90°$; and *L-stable* if it is A-stable and additionally $R(z) \to 0$ as $\mathrm{Re}(z) \to -\infty$, the latter condition being what damps very stiff transients out in a single step rather than carrying them along.

The initial-value problem (1) is called *stiff* on $[t_0, t_{\mathrm{end}}]$ when an explicit method is forced to take step sizes much smaller than what local accuracy alone would demand, in order to keep the dimensionless products $h\lambda_k$ inside the stability domain, where $\{\lambda_k\}$ are the eigenvalues of J along the trajectory. Stiffness is therefore not a property of the system in isolation; it depends on the spectrum of J, on the integration horizon, and on the requested tolerance. Chemical-kinetics networks with rate constants spanning several orders of magnitude are stiff in this sense, as are many discretised parabolic PDEs.

Stiff problems require methods whose stability domain contains the entire left half-plane (or a wide wedge thereof). Of the four methods listed above, BDF/NDF and Rosenbrock4 satisfy this requirement; Adams-Moulton and Tsit5 do not, and will perform poorly on stiff systems regardless of the requested tolerance.

2.3 Multistep methods

Linear multistep methods build the solution at the new time step x_n from a linear combination of the most recent $k+1$ states and right-hand-side values, $\{x_{n-j}, f_{n-j}\}_{j=0}^{k}$. They re-use information from earlier steps and so attain high accuracy with few right-hand-side evaluations per step, at the price of having to manage that history.

CppODE implements both the BDF and the Adams families through a common internal representation, due to Nordsieck and used throughout SUNDIALS [1]. The idea is to store, instead of the past $k+1$ states, a single *scaled derivative array*

$$z_n = \left(x_n,\, h_n \dot{x}_n,\, \frac{h_n^2}{2!}\ddot{x}_n,\, \ldots,\, \frac{h_n^k}{k!} x_n^{(k)} \right),$$

i.e. the value at the current time and its first k derivatives, each multiplied by the appropriate power of the current step size to make the entries dimensionally homogeneous. Changing the step size or the integration order then becomes a single linear transformation of z_n rather than a refit through the past data, which simplifies both the bookkeeping and the implementation of variable-order and variable-step control.

Backward Differentiation Formula (`bdf`)

The BDF formula of order q defines x_n implicitly by the condition

$$\sum_{j=0}^{q} \alpha_j \, x_{n-j} \;=\; h_n \, f(x_n, t_n), \tag{2}$$

which is exact whenever $x(t)$ is a polynomial of degree at most q on the past $q+1$ nodes. The method is A-stable for $q \le 2$ and A(α)-stable for $q \in \{3, 4, 5\}$; the stability angle α shrinks monotonically from 86° at $q = 3$ to 51° at $q = 5$.

The implicit corrector (2) is solved by Newton iteration,

$$W \Delta x^{(\nu)} = -\tilde{r}\left(x_n^{(\nu)}\right), \qquad x_n^{(\nu+1)} = x_n^{(\nu)} + \Delta x^{(\nu)}, \tag{3}$$

with leading coefficient $\beta_0 = 1/\alpha_0$ and the iteration matrix

$$W = \frac{1}{h_n \beta_0} \mathbb{1} - J(x_n^{(\nu)}, t_n), \tag{4}$$

in the form adopted by SUNDIALS [1], which differs from the textbook expression $I - h_n \beta_0 J$ by the overall scalar factor $h_n \beta_0$. The factorisation of W is the dominant cost per accepted step and is reused for as long as the contraction rate of the Newton iteration permits.

The default in CppODE is the Numerical Differentiation Formula (NDF) variant of [2], which adds a correction proportional to the Nordsieck predictor error to the BDF corrector,

$$\sum_{m=1}^{q} \frac{1}{m} \nabla^m x_n - h_n f(x_n, t_n) - \kappa_q \gamma_q \, (x_n - x_n^{\text{pred}}) = 0, \qquad \gamma_q = \sum_{m=1}^{q} \frac{1}{m}, \tag{5}$$

where ∇ denotes the backward-difference operator. The algebraic effect of the modification is to scale β_0 in (3) by a factor $(1 - \kappa_q)$ and to multiply the leading truncation-error coefficient $1/(q+1)$ by the factor $1 + \kappa_q (q+1) \gamma_q$. With $\kappa_q < 0$ the truncation error decreases at fixed step size while the iteration matrix is unchanged, which permits a larger step at a fixed accuracy requirement. The values of κ_q adopted by CppODE are tabulated in Appendix A; they reproduce the 25 % step-size gain over classical BDF reported by [2] for $q \le 4$, at the cost of a small reduction in stability angle. Setting `useNDF = FALSE` recovers the classical BDF formulas ($\kappa_q \equiv 0$).

A typical accepted step performs one right-hand-side evaluation for the predictor and between one and three right-hand-side evaluations together with the same number of linear solves for the corrector. A new Jacobian evaluation and LU factorisation are triggered only when the Newton iteration stalls or when the step size or order changes substantially. BDF/NDF is the default method in CppODE and the recommended choice when stiffness is suspected.

Adams-Moulton in PECE form (`adams`)

The Adams-Moulton family integrates the local interpolant of f over $[t_{n-1}, t_n]$. Writing f as the unique polynomial through the $q+1$ samples $\{(t_{n-j}, f_{n-j})\}_{j=0}^{q}$ and integrating gives

$$x_n \;=\; x_{n-1} \;+\; h_n \sum_{j=0}^{q} \beta_j \, f(x_{n-j}, t_{n-j}), \tag{6}$$

which is implicit in x_n through the $j = 0$ term. CppODE evaluates the corrector in predict-evaluate-correct-evaluate (PECE) mode rather than by Newton iteration. An explicit predictor (Adams-Bashforth of the same order q) first supplies a candidate value x_n^P for the new state; the right-hand side is then evaluated at x_n^P; the corrector (6) is evaluated explicitly with $f(x_n^P, t_n)$ in place of $f(x_n, t_n)$, yielding a refined x_n; and a final right-hand-side evaluation at the refined state is entered into the Nordsieck history. The procedure costs two right-hand-side evaluations per accepted step, with no nonlinear iteration and no linear solve.

The Adams-Moulton stability domain on the negative real axis shrinks rapidly with order; for $q \ge 3$ it does not contain even a small wedge of $\mathbb{C}^-$ away from the origin, so the method is unsuitable for stiff problems. For non-stiff problems with smooth solutions and a long integration horizon, however, the high attainable order ($q_{\max} = 12$) and the cost of two right-hand-side evaluations per accepted step make it competitive with explicit Runge-Kutta methods of fixed order.

2.4 Single-step methods

Single-step methods carry no information beyond the current state. They achieve order $p > 1$ by evaluating internal stages within each step rather than by reaching back into the integration history. The absence of history makes restarts after events trivial; both methods below restart at full order in a single step, unlike the multistep methods, which must rebuild the Nordsieck history at order 1.

Rosenbrock (`rb4`)

A Rosenbrock method of s stages with diagonal coefficient γ linearises the implicit Runge-Kutta condition about (x_n, t_n) and defines stage values k_i by

$$W k_i = f\left(x_n + \sum_{j<i} \alpha_{ij}\, k_j,\; t_n + c_i\, h_n\right) + \frac{1}{h_n} \sum_{j<i} \frac{\gamma_{ij}}{\gamma}\, k_j + h_n\, d_i\, \frac{\partial f}{\partial t}(x_n, t_n), \tag{7}$$

with the iteration matrix

$$W = \frac{1}{\gamma\, h_n} \mathbb{1} - J(x_n, t_n). \tag{8}$$

The method coefficients are listed in Table 2.

Symbol	Meaning
α_{ij}	weights with which stage i combines all earlier stages k_j, $j < i$
c_i	time abscissa of stage i inside the step, with $c_i = \sum_{j<i} \alpha_{ij}$
γ_{ij}	weights with which the linear-solve correction at stage i uses earlier stages
γ	common diagonal coefficient appearing in the iteration matrix W
d_i	weights that pick up the explicit time dependence of f via $\partial f/\partial t$
m_i	weights combining the stages into the advanced solution x_{n+1}
$\hat{m}_i$	weights combining the stages into the embedded order-3 estimator used for step control

Table 2: Rosenbrock method coefficients.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + \sum_{i=1}^{s} m_i\, k_i, \qquad \hat{x}_{n+1} \;=\; x_n + \sum_{i=1}^{s} \hat{m}_i\, k_i.$$

Each stage (7) is a single linear solve; no Newton iteration is required. Per accepted step CppODE evaluates J and factorises W once, performs five right-hand-side evaluations, and back-substitutes six times.

CppODE uses the order-4 coefficients of [3, Sec. IV.7], the same set as the `rosenbrock4` stepper of Boost.Numeric.Odeint. The pair is L-stable and carries an embedded order-3 estimator. Continuous output on $[t_n, t_{n+1}]$ is delivered by an order-3 interpolant constructed from the stage values k_i and two additional coefficient sets $d_i^{(2)}$ and $d_i^{(3)}$ tabulated in Appendix C; no extra right-hand-side evaluations are required for output points.

Rosenbrock methods are competitive with BDF on stiff problems whose Jacobian is cheap relative to the right-hand side, and they avoid the order-rebuilding cost incurred by the multistep methods after an event. Rosenbrock4 is therefore the recommended stiff method for systems with frequent state discontinuities at moderate state dimension.

Tsitouras (`tsit5`)

Tsitouras is the explicit Runge-Kutta pair of order 5(4) derived by [4] under the simplifying assumption that the first column of the coefficient matrix vanishes, $a_{i,1} = 0$ for $i = 2, \dots, s$. Removing this column reduces the number of order conditions to be satisfied during construction and was shown to admit a particularly favourable choice of nodes c_i in terms of leading-error norm. The pair has $s = 7$ stages with the First-Same-As-Last (FSAL) property,

$$k_7 \;=\; f(x_{n+1},\, t_n + h_n) \;=\; k_1 \text{ at step } n+1,$$

so an accepted step costs six right-hand-side evaluations rather than seven. The method coefficients are listed in Table 3.

Symbol	Meaning
a_{ij}	weights with which stage i combines earlier stages (lower triangular; first column is zero)
c_i	time abscissa of stage i inside the step, with $c_i = \sum_{j<i} a_{ij}$
b_i	weights combining the stages into the advanced order-5 solution; $b_7 = 0$
$\hat{b}_i$	weights combining the stages into the embedded order-4 estimator; $\hat{b}_7 \neq 0$
k_i	right-hand side at stage i: $k_i = f\Big(x_n + h_n \sum_{j<i} a_{ij} k_j,\ t_n + c_i h_n\Big)$

Table 3: Tsit5 method coefficients.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + h_n \sum_{i=1}^{7} b_i\, k_i, \qquad \hat{x}_{n+1} = x_n + h_n \sum_{i=1}^{7} \hat{b}_i\, k_i,$$

and the local error estimate used for step-size control is

$$e_{n+1} = h_n \sum_{i=1}^{7} (\hat{b}_i - b_i)\, k_i.$$

Continuous output is produced by a cubic Hermite interpolant on $[t_n, t_{n+1}]$, using the accepted-step endpoints (x_n, x_{n+1}) and their derivatives $(f_n, f_{n+1}) = (k_1, k_7)$, both already in cache. No additional right-hand-side evaluations are required for output points; the interpolation order ($p = 4$ globally, $p = 3$ locally on the interval) is one less than the order of the underlying step.

Tsitouras has no Jacobian and no linear solve and performs the largest amount of integration per right-hand-side call of the four methods considered here. It is the appropriate choice for non-stiff problems whose right-hand side is expensive to evaluate.

2.5 Adaptive step-size control

The error estimate e_n delivered by each method is reduced to a scalar through the weighted root-mean-square norm

$$\|e_n\|_{\mathrm{WRMS}} = \sqrt{\frac{1}{n_x} \sum_{j=1}^{n_x} \left(\frac{e_{n,j}}{w_{n,j}} \right)^2}, \qquad w_{n,j} = \texttt{atol} + \texttt{rtol}\, |x^{\mathrm{ref}}_{n,j}|, \tag{9}$$

in which $w_{n,j}$ is a per-component tolerance combining an absolute floor `atol` and a relative scale `rtol` $|x^{\mathrm{ref}}_{n,j}|$. Normalising $e_{n,j}$ by $w_{n,j}$ before squaring places components of widely differing magnitudes on a common scale and removes the dependence of the acceptance test on the physical units of the state. The reference state x^{ref}_n that enters the relative term is the Nordsieck predictor in BDF/NDF and the component-wise maximum $\max(|x_n|, |x_{n+1}|)$ in the single-step methods; the two choices are both standard and produce numerically indistinguishable values of the norm in practice. A step is accepted when $\|e_n\|_{\mathrm{WRMS}} \leq 1$.

The next step size h_{n+1} is selected by a control law of the type analysed in [3], applied to the WRMS norm of the current step and, when active, the WRMS norm of the previous step,

$$h_{n+1} = h_n \min\Big\{ \rho_{\max}, \max\Big(\rho_{\min}, S\, \|e_n\|_{\mathrm{WRMS}}^{-1/(p+1)} \|e_{n-1}\|_{\mathrm{WRMS}}^{\beta/(p+1)} \Big) \Big\},$$

with safety factor $S < 1$, growth limits $\rho_{\min}, \rho_{\max}$, and a memory coefficient $\beta \in [0, 1]$. The case $\beta = 0$ is the standard integral or I-controller, in which h_{n+1} depends on the current error norm only; CppODE uses this form for the multistep methods, where the Nordsieck representation already smooths the underlying signal. The case $\beta > 0$ is the proportional-integral (PI) controller of [3]; the single-step methods adopt this form to damp the high-frequency oscillations in h_n that a pure I-controller can otherwise produce on stiff or near-singular problems.

2.6 Dense output

User-requested output times do not in general coincide with the adaptive step grid produced by the controller. Each accepted step therefore carries a local interpolant that allows evaluation of $x(t)$ at any time inside that step at the cost of polynomial arithmetic only. CppODE uses Hermite interpolation, which constrains both values and derivatives of the interpolant at the step endpoints. This form attains the same order as the underlying step without recourse to data outside the step: state and right-hand side at the two endpoints, augmented by stage values for the single-step methods. No additional right-hand-side evaluations are required for dense output. The local interpolation order is 4 for Rosenbrock4 and Tsit5; the multistep methods reuse the Nordsieck history, which carries derivatives up to order q.

2.7 Automatic Forward sensitivities

Dual-number arithmetic

Parameter sensitivities of the trajectory are computed by forward-mode automatic differentiation rather than by finite differences (whose truncation and round-off errors require multiple integrations to control) or by symbolic derivation of the sensitivity equations (whose intermediate expressions grow rapidly with the dimension of f). Forward-mode AD augments every floating-point scalar in the computation with an additional component that records its directional derivative with respect to a chosen parameter direction. The arithmetic operations and elementary functions are overloaded to act on this augmented type so that the chain rule is applied automatically; the C++ source that evaluates f is then compiled unchanged for both the primal and the sensitivity computation.

The augmented type is the *dual number*

$$\tilde{x} = x + x'\,\varepsilon, \qquad \varepsilon^2 = 0, \quad \varepsilon \neq 0,$$

where $x \in \mathbb{R}$ is the primal value and $x' \in \mathbb{R}$ is the directional derivative. Treating $\tilde{x}$ as the first two terms of a Taylor expansion in ε and discarding all higher powers (which vanish by $\varepsilon^2 = 0$), the arithmetic of dual numbers follows from the standard rules of calculus: for $\tilde{x} = x + x'\varepsilon$ and $\tilde{y} = y + y'\varepsilon$,

$$\tilde{x} + \tilde{y} = (x + y) + (x' + y')\varepsilon, \qquad \tilde{x} \cdot \tilde{y} = xy + (xy' + x'y)\varepsilon,$$

and for any smooth scalar function g,

$$g(\tilde{x}) = g(x) + g'(x)\,x'\,\varepsilon.$$

The chain rule is therefore intrinsic to the algebra: any composition of dual-number operations automatically tracks the derivative of the composed function, and no code transformation or symbolic manipulation is required. The same source that evaluates f over $\mathbb{R}$ evaluates both f and $\partial f/\partial x$ when called with dual-number inputs, by virtue of the overloaded operators and elementary functions.

Sensitivity equations via dual evaluation

Let the ODE right-hand side depend on a parameter vector $p \in \mathbb{R}^{n_p}$,

$$\dot{x} = f(x, p, t), \qquad x(t_0) = x_0(p),$$

and define the *sensitivity matrix*

$$S_p = \frac{\partial x}{\partial p} \in \mathbb{R}^{n_x \times n_p}, \qquad (S_p)_{ij} = \frac{\partial x_i}{\partial p_j}.$$

To derive the evolution equation for S_p via dual arithmetic, seed each state component with its row of S_p and form the dual-valued state vector

$$\tilde{x}_i = x_i + (S_p)_{ij}\,\varepsilon_j,$$

where ε_j is the dual unit associated with direction p_j and summation over j is implied. Evaluating f component-wise over the dual algebra and collecting the ε_j-coefficient of the i-th output gives

$$[f(\tilde{x}, p, t)]_{i,\,\varepsilon_j} = \frac{\partial f_i}{\partial x_k}\,(S_p)_{kj} + \frac{\partial f_i}{\partial p_j},$$

where repeated k is summed. Since the primal part of the dual evaluation recovers $\dot{x}_i = f_i(x, p, t)$, differentiating $\dot{x}_i = f_i$ with respect to p_j and identifying the ε_j-coefficient with $(\dot{S}_p)_{ij}$ yields

$$\dot{S}_p = \frac{\partial f}{\partial x}\,S_p + \frac{\partial f}{\partial p}, \qquad S_p(t_0) = \frac{\partial x_0}{\partial p}. \tag{10}$$

Hence a single dual-number evaluation of f propagates the full sensitivity matrix S_p in lockstep with the primal state x, at a cost proportional to n_p. No additional Jacobian evaluations are required; the same code path that computes f computes $(\partial f/\partial x)\,S_p + \partial f/\partial p$ automatically.

Seeding: from S_p to S_θ

In many applications the natural parameters of inference are not the raw entries of p but a higher-level vector $\theta \in \mathbb{R}^M$ to which both the dynamic parameters p_{dyn} and the initial condition $x_0 = p_{\text{init}}(\theta)$ are linked through a user-defined reparametrisation $p = \Phi(\theta)$. The quantity of interest is then

$$S_\theta = \frac{\partial x}{\partial \theta} \in \mathbb{R}^{n_x \times M},$$

related to S_p by the chain rule $S_\theta = S_p\,\partial p/\partial\theta$. Substituting this relation into (10) yields the evolution equation for S_θ directly,

$$\dot{S}_\theta = \frac{\partial f}{\partial x}\,S_\theta + \frac{\partial f}{\partial p_{\text{dyn}}}\,\frac{\partial p}{\partial \theta}, \qquad S_\theta(t_0) = \frac{\partial p_{\text{init}}}{\partial \theta}. \tag{11}$$

The matrix $\partial p/\partial\theta$ is the *seed matrix*, supplied by the user (through the `sens1ini` argument of `solveODE()`) and held fixed for the duration of the integration. The choice $\partial p/\partial\theta = I_{n_p}$ recovers the canonical case $S_\theta = S_p$; any other choice carries the sensitivity computation in M specified directions of θ-space at the same cost as integrating M additional states alongside x.

Dual-number aware error norm

When forward sensitivities are active, the integrator carries the sensitivity matrix $S_\theta \in \mathbb{R}^{n_x \times M}$ alongside the primal state. Every accepted step then produces $1 + M$ error vectors of length n_x: the primal estimate $e_n^{(x)} \in \mathbb{R}^{n_x}$ and a sensitivity estimate $e_S^{(k)} \in \mathbb{R}^{n_x}$ for each column $\partial x/\partial\theta_k$ of S_θ. Two natural ways to combine these into a single scalar suggest themselves: a flat WRMS that pools all $n_x(1+M)$ components into one sum, or a maximum over per-vector WRMS norms. CppODE adopts the second, following the staggered-tolerance convention of CVODES (the `CV_STAGGERED` mode of its sensitivity solver).

The state norm $\|e_n^{(x)}\|_{\text{WRMS}}$ is exactly (9). The sensitivity column norm is its analogue, with S_θ replacing x in both the error vector and the weight,

$$\|e_S^{(k)}\|_{\text{WRMS}} = \sqrt{\frac{1}{n_x}\sum_{i=1}^{n_x}\left(\frac{(e_S)_{ik}}{w_i^{(k)}}\right)^2}, \qquad w_i^{(k)} = \texttt{atol} + \texttt{rtol}\,|(S_\theta)_{ik}^{\text{ref}}|, \tag{12}$$

in which $(S_\theta)_{ik}^{\text{ref}}$ is the reference value of the i-th component of column k, taken from the same source (predictor or pre/post maximum) as x_n^{ref} for the state. The scalar that the step-size controller sees is the worst of the state norm and the M column norms,

$$\|e_n\|_{\text{WRMS}} = \max\left\{\|e_n^{(x)}\|_{\text{WRMS}},\ \max_{1\le k\le M}\|e_S^{(k)}\|_{\text{WRMS}}\right\}. \tag{13}$$

The maximum-of-norms convention is preferred to the flat WRMS, which divides by the total component count $n_x(1+M)$ and therefore averages the squared errors across directions in θ: a large error in one sensitivity column k_0 can be obscured by M small errors in the remaining columns and the state, leading the integrator to accept a step that is too large in the direction k_0. The maximum norm prevents this dilution and forces each direction in θ to satisfy the requested tolerance independently. Because the controller still operates on a single scalar, the cost relative to the flat norm is a constant overhead in the number of accepted steps, and the control-law parameters $(S, \rho_{\min}, \rho_{\max}, \beta)$ require no adjustment.

The same WRMS structure governs the Newton convergence test in BDF/NDF. A Newton iterate is accepted when the iteration correction satisfies (13), with the maximum again taken over the state and the per-column sensitivity components. Convergence in the primal therefore does not allow a loose solve in any sensitivity direction; a parameter direction that drives the correction norm up triggers either further Newton iterations or a step-size reduction, even when the primal residual is already small.

2.8 Events and restarts

CppODE supports two kinds of events that interrupt smooth integration. A *time-triggered event* fires at a user-specified time $t_e(\theta)$ that may depend on θ through a closed-form expression. A *root-triggered event* fires when a user-supplied event function $g_e(x,t)$ crosses zero along the trajectory; the firing time $t^*(\theta)$ is then defined implicitly by $g_e(x(t^*), t^*) = 0$. In both cases an event applies a discontinuous reset to the state,

$$x^+ = g(x^-, t),$$

typical examples being instantaneous dosing of a kinetic species, a clipping rule that enforces non-negativity, or a switch between dynamical regimes. The remainder of this section first describes the cost of restarting the integrator after a discontinuity, then sets up the sensitivity-transport problem, and finally derives the correction for each event type.

Restart cost

Single-step methods (Rosenbrock4, Tsit5) carry no information beyond the current state and resume integration in a single full-order step after the reset. Multistep methods (BDF/NDF, Adams) build their order through history and must restart at order 1, rebuilding the Nordsieck array from scratch over several steps. This restart penalty is the dominant cost on problems with frequent events and is the reason Rosenbrock4 is the recommended stiff method when discontinuities are common, even though BDF/NDF is the default elsewhere.

Sensitivity transport across a discontinuity

The reset map g acts on the trajectory $x(t,\theta)$ at the firing time $t^*(\theta)$. The naive update of the sensitivity, by the linearisation of g around the pre-event state, $S_\theta^+ = (\partial g/\partial x)\, S_\theta^-$, is incorrect in the presence of an event because the firing time itself depends on θ. A perturbation δt^* of the firing time induces short integrations of $\dot{x} = f$ on either side of the event, with f taking different values before and after the reset. Denote by f^- the right-hand side just before the reset, evaluated at x^-, and by f^+ the right-hand side just after the reset, evaluated at $g(x^-, t^*)$. The sensitivity at the first grid time following the event then receives two contributions in addition to the linearised reset:

1. *Pre-event drift.* The pre-event trajectory x^- flows along $\dot{x} = f^-$ from the grid time $t_*^{(0)} := \mathrm{scalar}(t^*)$ to the θ-perturbed firing time t^*, over a displacement $\delta t^* = t^* - t_*^{(0)}$ whose scalar part vanishes but whose θ-derivatives do not.
2. *Post-event drift.* The reset state $g(x^-, t^*)$ then flows along $\dot{x} = f^+$ from t^* back to the grid time over the negative of the same displacement.

Both shifts contribute terms of order $f^{\pm}\,(\partial t^*/\partial\theta)$ to the sensitivity at the grid time. The correct first-order update reproduces a closed-form expression known as the *saltation matrix*, recovered below as a consequence of the construction.

CppODE realises this correction by replacing the bare reset with a forward-drift-reset-backward-drift sequence, evaluated in dual-number arithmetic. Each drift is integrated by Heun's method, a two-stage second-order

Runge-Kutta scheme also known as the improved Euler method, which averages the right-hand side at the start and the end of the interval. Applied to dual-typed input the average is a pure arithmetic operation: the scalar component of x at the grid time is unchanged (since the displacement δt^* has scalar part zero), but the θ-derivatives of the displacement propagate into the state. A single Euler step would suffice for first-order sensitivities, but its truncation error of order δt^{*2} contributes a non-vanishing second-order θ-derivative through the dual algebra and would distort the Hessian. Heun's method has truncation error $O(\delta t^{*3})$ and recovers the correct second-order sensitivities. The reset map g is applied between the two drifts in the same dual-number algebra, so its first- and second-order derivatives propagate automatically.

When AD is disabled (pure-double integration), no transport is required: g is applied to the state as an ordinary function, and the remainder of this section may be skipped.

The construction differs in detail between time-triggered and root-triggered events, because the firing-time perturbation δt enters the dual evaluation differently: directly in the time-triggered case, where $t_e(\theta)$ is supplied by the user and its dual derivatives propagate through the chain rule, and implicitly in the root case, where $t^*(\theta)$ must be reconstructed from the condition $g_e = 0$ via the implicit function theorem.

Time-triggered events

When the firing time is supplied directly as a closed-form expression $t_e(\theta)$, the chain-rule derivatives of t_e with respect to θ are immediately available. The integrator advances on the scalar grid time $t_e^{(0)} := \mathrm{scalar}(t_e)$ and treats the small displacement

$$\delta t = t_e - t_e^{(0)}, \qquad \mathrm{scalar}(\delta t) = 0, \tag{14}$$

as a dual number whose value is zero but whose first-order tangents are $\partial t_e/\partial\theta_a$ and whose second-order tangents are $\partial^2 t_e/\partial\theta_a\partial\theta_b$, both inherited automatically from the user-supplied expression for t_e. The state at the grid time $t_e^{(0)}$ is denoted x^- and is itself a dual number that carries the pre-event sensitivities S_θ in its first-order tangent slots.

The transport across the event consists of three steps: a short Heun integration along $\dot{x} = f$ from $t_e^{(0)}$ forward to t_e, the reset map g applied at t_e, and a short Heun integration backward from t_e to $t_e^{(0)}$,

$$\begin{aligned}
&\text{forward:} \quad f_1 = f(x^-, t_e^{(0)}), \quad x_E = x^- + f_1\,\delta t, \quad f_2 = f(x_E, t_e),\\
&\qquad\qquad x^* = x^- + \tfrac{1}{2}(f_1 + f_2)\,\delta t,\\
&\text{reset:} \quad x_+^* = g(x^*, t_e),\\
&\text{backward:} \quad \tilde{f}_1 = f(x_+^*, t_e), \quad x_B = x_+^* - \tilde{f}_1\,\delta t, \quad \tilde{f}_2 = f(x_B, t_e^{(0)}),\\
&\qquad\qquad x^+ = x_+^* - \tfrac{1}{2}(\tilde{f}_1 + \tilde{f}_2)\,\delta t.
\end{aligned} \tag{15}$$

The forward leg evaluates the right-hand side at both endpoints of the short interval ($t_e^{(0)}$ and t_e); averaging the two values is the standard Heun update and produces a result accurate to second order in δt. The reset map g is then applied on the event surface at t_e, and the backward leg is the same Heun rule shifted from t_e back to the grid time, this time using the post-event right-hand side.

A single Euler step would suffice for first-order sensitivities: since $\mathrm{scalar}(\delta t) = 0$, the powers δt^k for $k \geq 2$ have zero first-order dual content, and any $O(\delta t^2)$ term drops out of $\partial x^+/\partial\theta$. For second-order sensitivities, however, the curvature term

$$\tfrac{1}{2}(J\,f + \partial f/\partial t)\,\delta t^2, \qquad J = \partial f/\partial x,$$

acquires a non-vanishing second-order dual component, because the square of δt has a non-zero mixed derivative

$$\frac{\partial^2(\delta t)^2}{\partial\theta_a\,\partial\theta_b} = 2\,\frac{\partial t_e}{\partial\theta_a}\,\frac{\partial t_e}{\partial\theta_b},$$

despite both its value and its first derivatives being zero. The Heun midpoint absorbs this curvature term through the second right-hand-side evaluation at the shifted state *and* the shifted time. The use of the

dual-typed time argument t_e rather than $t_e^{(0)}$ alone in f_2 and $\tilde{f}_2$ is therefore essential: omitting it would discard the $\frac{1}{2}(\partial f/\partial t)\,\delta t^2$ contribution for systems with explicit time dependence, even though the time shift itself is invisible at the scalar level ($\mathrm{scalar}(\delta t) = 0$).

Root-triggered events

For a root event the firing time $t^*(\theta)$ is defined implicitly by

$$G(t^*(\theta), \theta) := g_e(x(t^*(\theta), \theta), t^*(\theta)) = 0. \tag{16}$$

Three steps lead from this condition to the corrected state at the next grid time: localisation of t^* on the scalar trajectory, reconstruction of the dual residual δt^* via the implicit function theorem, and the same forward-reset-backward Heun transport as in the time-triggered case.

Localisation The integrator detects a root by tracking the sign of g_e at successive accepted steps: when the sign of $g_e(x_n, t_n)$ and $g_e(x_{n+1}, t_{n+1})$ disagree, a sign change must have occurred on $[t_n, t_{n+1}]$. The crossing is then refined by bisection using the dense output of the accepted step, which means no extra right-hand-side evaluations are spent on the localisation itself. The bisection terminates when the bracket width falls below a user-supplied root tolerance or below an integrator-determined floor; the midpoint of the final bracket is returned as $t_*^{(0)} := \mathrm{scalar}(t^*)$. The dual residual $\delta t^* = t^* - t_*^{(0)}$ must now be reconstructed from (16), since bisection on the scalar trajectory provides no information about how t^* depends on θ.

The dual residual via the implicit function theorem The implicit function theorem yields the derivative of $t^*(\theta)$ directly from the constraint $G(t^*, \theta) = 0$, without explicit inversion, provided G is smooth at the crossing and $\partial G/\partial t \neq 0$ there. Differentiating (16) once with respect to θ and applying the chain rule gives

$$\frac{\partial t^*}{\partial \theta} = -\frac{G_\theta}{G_t} = -\left.\frac{\nabla_x g_e \cdot S_\theta}{\nabla_x g_e \cdot f + \partial g_e/\partial t}\right|_*, \tag{17}$$

in which the denominator is the total time derivative $\dot{g}_e = \nabla_x g_e \cdot f + \partial g_e/\partial t$ along the trajectory at the crossing. The non-vanishing-denominator condition $\dot{g}_e \neq 0$ is the geometric statement that the trajectory crosses the event surface transversally rather than grazing it; for grazing crossings the firing time itself is not differentiable in θ and a separate fallback applies, see the discussion of simultaneous events below. In dual-number form, (17) is realised as the quotient

$$\delta t^* = -\frac{g_e(x^-, t_*^{(0)})}{\dot{g}_e(x^-, t_*^{(0)})}, \tag{18}$$

where both g_e and $\dot{g}_e$ are evaluated on the dual-typed state x^- at the scalar grid time. The codegen-emitted partials $\nabla_x g_e$ and $\partial g_e/\partial t$ are required only for the assembly of $\dot{g}_e$; the dual chain rule propagates $\nabla_x g_e \cdot S_\theta$ through $g_e(x^-, t_*^{(0)})$ automatically. The scalar residual of the numerator (the bisection localisation tolerance) is set to zero before the division so that the value of δt^* is exactly zero by construction; only its dual derivatives remain.

The first-order dual content of (18) reproduces (17) exactly. At second order the dual quotient misses one term, because g_e is evaluated at the fixed scalar time $t_*^{(0)}$ rather than at the dual-typed time t^*: the quotient rule cannot supply the implicit G_{tt}-contribution that would arise from differentiating g_e in its time slot. Expanding (16) to second order in θ and matching coefficients identifies the missing piece. Writing $\delta t^* = \delta t^*_{(1)} + \delta t^*_{(2)} + \cdots$ and using $G(t_*^{(0)}, \theta^{(0)}) = 0$,

$$0 = G_t\,\delta t^*_{(2)} + \tfrac{1}{2} G_{tt}\,(\delta t^*_{(1)})^2 + G_{t\theta}\,\delta t^*_{(1)}\,\delta\theta + \tfrac{1}{2} G_{\theta\theta}\,\delta\theta^2,$$

of which the dual quotient supplies the $G_{t\theta}$- and $G_{\theta\theta}$-contributions but not the G_{tt}-contribution. The corrected update is therefore

$$\delta t^* \leftarrow \delta t^* - \tfrac{1}{2}\,\frac{G_{tt}}{G_t}\,(\delta t^*)^2, \tag{19}$$

where

$$G_{tt} = \frac{\mathrm{d}^2}{\mathrm{d}t^2} g_e(x(t), t)\Big|_*$$

is the second total time derivative of g_e along the trajectory. CppODE supplies G_{tt} either through a codegen-emitted analytic callback or, when unavailable, through a forward finite difference of $\dot{g}_e$. Equation (19) touches only the second-order dual components of δt^*; the value and first-order parts are unchanged.

Forward-reset-backward sandwich With δt^* in hand, the same forward-reset-backward Heun procedure of (15) carries x^- across the event to the grid time $t_*^{(0)}$, with δt replaced by δt^* and the event time t_e replaced by $t_*^{(0)} + \delta t^*$. The five right-hand-side evaluations and the dual-typed time arguments play exactly the same role as in the time-triggered case.

Simultaneous events Two or more root conditions can cross zero within the same accepted step. CppODE localises each crossing independently and groups any that share the same $t_*^{(0)}$ to within the localisation tolerance into a simultaneous-event cluster. For such a cluster the forward Heun shift is performed once to the common event surface, every triggered reset is applied at that surface (the reset maps are assumed to commute), and a single backward Heun shift returns the state to the grid time. The cost is four right-hand-side evaluations independent of the number of simultaneous events. The dual residual δt^* is computed once from the first event in the cluster with non-zero $\dot{g}_e$ at the surface; the remaining events share the same δt^*, since they all sit on the same grid time with the same parameter dependence inherited through x^-.

If every triggered event in the cluster has $\dot{g}_e = 0$ at the surface (a tangential crossing of all event functions), the denominator in (17) vanishes and the implicit function theorem fails to deliver $\partial t^*/\partial\theta$: the firing time is then insensitive to θ at first order. CppODE detects this case by the smallness of $|\dot{g}_e|$ and falls back to applying the reset as a plain map without the saltation transport, which is mathematically correct in this limit because the two saltation contributions $f^{\pm}\,(\partial t^*/\partial\theta)$ both vanish.

Connection to the saltation matrix

The standard treatment of sensitivity transport across a discontinuity yields a closed-form correction, the *saltation matrix*, which generalises the bare reset Jacobian $\partial g/\partial x$ by an additive term accounting for the θ-dependence of the firing time:

$$S_\theta^+ = \frac{\partial g}{\partial x}\, S_\theta^- + \left(\frac{\partial g}{\partial t} + \frac{\partial g}{\partial x}\, f^- - f^+\right) \frac{\partial t^*}{\partial \theta}.$$

The first-order content of the dual-number forward-drift-reset-backward-drift construction reproduces this expression exactly, both for time-triggered events ($\partial t^*/\partial\theta = \partial t_e/\partial\theta$ supplied directly through the user expression for $t_e(\theta)$) and for root events ($\partial t^*/\partial\theta$ computed from (17)). The motivation for adopting the implicit-function and dual-number formulation in CppODE is that it extends naturally to second-order sensitivities, for which the classical saltation-matrix form has no closed-form analogue in the literature. The implementation resides in `cppode_integrate_times.hpp`.

2.9 Method selection

The recommended choice of method for the most common scenarios is summarised in Table 4.

The cost of an inappropriate choice is asymmetric. An explicit method applied to a stiff problem will fail or be driven to step sizes that make the integration uneconomic, while an implicit method applied to a non-stiff problem incurs only a constant overhead per step. In the absence of prior knowledge of the spectrum of J, BDF/NDF is the safer default.

Situation	Recommended method
Default; stiffness unknown	`"bdf"`
Stiff, frequent events	`"rb4"`
Stiff, smooth, large n_x with sparse Jacobian	`"bdf"` with `sparse = TRUE`
Non-stiff, expensive right-hand side	`"tsit5"`
Non-stiff, cheap right-hand side, long horizon, high accuracy	`"adams"`

Table 4: Recommended `method` argument for common integration scenarios. Stiffness is the dominant factor; in its absence, right-hand-side cost and required accuracy decide.

3 `CVODE()`

The `CVODE()` entry point compiles a model against the system-installed SUNDIALS CVODE/CVODES library and exposes the result through the same `solveODE()` interface as `CppODE()`. The two backends therefore differ only in the numerical core; the codegen pipeline, the `events`, `forcings`, `rootfunc`, and `fixed` arguments, the shape of the returned object, and the column ordering of `parms` and `sens1ini` all behave identically. SUNDIALS (≥ 6.0) must be installed on the build host; KLU (used for sparse Jacobians) is detected the same way and is required when `sparse = TRUE`.

3.1 What is shared

The symbolic Jacobian $J = \partial f/\partial x$ is derived through SymPy at code-generation time and emitted as analytic C++ exactly as in `CppODE()`. The KLU sparse linear solver path activates automatically when the Jacobian is sufficiently sparse and the SuiteSparse libraries were detected at install time; the dense fallback uses LAPACK. Forcing functions are PCHIP-interpolated, events apply a discontinuous reset to the state, and root events fire when a user-supplied scalar function crosses zero.

3.2 What differs

Sensitivity right-hand side. CVODES integrates the analytic forward-sensitivity equation

$$\dot{S}_k = J(x,p)\, S_k + \frac{\partial f}{\partial p_k}, \qquad k = 1, \dots, n_p, \tag{20}$$

directly. The in-tree backend recovers the same equation by evaluating f on dual-typed state, cf. (10). The two formulations are mathematically equivalent. Numerically, the analytic right-hand side avoids the per-evaluation arena allocation required by the dual-number path and is preferable on large problems with a sparse $\partial f/\partial p$; conversely, the dual-number path benefits from SymPy's common-subexpression elimination on small problems for which integrator control flow dominates the cost.

Available methods. CVODE provides only the multistep families. Calling `CVODE(method = "bdf")` or `CVODE(method = "adams")` selects between them; the single-step methods `rb4` and `tsit5` are not available through this backend. The cost-per-step accounting and the stability properties documented for the multistep methods in Section 2 therefore carry over directly.

Second-order sensitivities. CVODES exposes only first-order forward sensitivities; the nested-dual construction has no counterpart in the SUNDIALS API. `CVODE()` therefore does not accept `deriv2 = TRUE`.

Saltation transport across events. Events on `CVODE()` models use the same forward-drift-reset-backward-drift sequence as `CppODE()`, with the first-order saltation correction of equations (15) and (17). The transport is performed in the generated glue layer rather than inside CVODES, since CVODES exposes no entry point at which the sensitivity solver can be informed of the discontinuity.

Reparametrisation seeding. The chain-rule construction $S_\theta = S_p\, \partial p/\partial\theta$ of equation (11) is identical to the `CppODE()` case. `solveODE()` accepts the same `sens1ini` matrix shapes (legacy, full, and partial) and the same row-name conventions; only second-order S_θ is unavailable.

3.3 When to choose which backend

`CVODE()` is appropriate when bit-for-bit reproducibility against the SUNDIALS reference implementation is required, when the deployment environment can satisfy the SUNDIALS dependency, or when the SUNDIALS KLU path is preferred over the in-tree sparse solver on large sparse problems. `CppODE()` is appropriate when the package must be self-contained, when second-order sensitivities are required, or when the single-step methods are required (Tsit5 for non-stiff problems with expensive right-hand sides; Rosenbrock4 for stiff problems with frequent events).

4 `funCpp()`

The `funCpp()` entry point compiles a system of algebraic expressions

$$y_i = g_i(x_1, \dots, x_{n_x}, p_1, \dots, p_{n_p}), \qquad i = 1, \dots, n_y, \tag{21}$$

together with optional first- and second-order derivatives. No time integration is performed: each call evaluates g at one or more rows of (x, p) values and returns the corresponding outputs, Jacobians, and Hessians as requested. Two use cases motivate the function. The first is the evaluation of observation maps $y = g(x, p)$ that connect integrator output to data in likelihood-based inference; the mapping is applied at every observation point and its Jacobian enters the gradient of the log-likelihood. The second is the evaluation of reparametrisation Jacobians $\partial p/\partial\theta$ that supply the seed matrix in equation (11) when the dynamic model is fitted in a transformed parameter space.

4.1 Unified API and two computational paths

Both values of the `derivMode` argument expose the same R-side methods: `func`, `jac`, `hess`, and the combined `evaluate`. They differ only in how the underlying C entry points are generated and dispatched.

- `"symbolic"`: the analytic Jacobian and Hessian of g are derived by SymPy at code-generation time. The compiled module exposes `_eval`, `_jacobian` (when `deriv`), and `_hessian` (when `deriv2`). When seeds are supplied, the chain rule is applied by BLAS-3 DGEMM through two `extern "C"` wrappers `_chain_jac` and `_chain_hess`, which delegate to the inline routines in `cppode_chain_blas.hpp`.
- `"dual"` (default): no SymPy step. The compiled module exposes `_eval`, `_eval_ad` (when `deriv`, on `cppode::dual<double, 0>`), and `_eval_ad2` (when `deriv2`, on the nested-dual alias `cppode::dual2nd<double, 0>`). The chain rule is applied inside the AD evaluation by seeding the dual-number tangent slots; the same source that evaluates g over $\mathbb{R}$ then also evaluates the chain-ruled $\partial y/\partial\theta$ and $\partial^2 y/\partial\theta^2$.

The two paths are equivalent in exact arithmetic. The dual path avoids the SymPy expansion of $\partial g/\partial p$ and $\partial^2 g/\partial p^2$, whose intermediate expressions can grow sharply with the depth of the compositions in g. The symbolic path benefits from SymPy's common-subexpression elimination when the analytic derivatives remain compact.

Call	dual path	symbolic path
`func(x, p)`	`_eval`	`_eval`
`jac(x, p)` (no seeds)	`_eval_ad` with identity seed	`_jacobian`
`jac(x, p, dX, dP)`	`_eval_ad` with user seeds	`_jacobian + _chain_jac`
`hess(x, p)` (no seeds)	`_eval_ad2` with identity seed	`_hessian`
`hess(x, p, dX, dP[, dX2, dP2])`	`_eval_ad2` with all seeds	`_hessian + _chain_hess`
`evaluate(…, deriv2 = FALSE)`	one `_eval_ad`-pass	`_eval + _jacobian` (+ chain)
`evaluate(…, deriv2 = TRUE)`	one `_eval_ad2`-pass	`_eval + _jacobian + _hessian` (+ chain)

Table 5: Dispatch within `funCpp()`. Both modes accept the same arguments and return arrays of the same shape; only the C entries called and the location of the chain-rule contraction differ.

The combined entry point `evaluate()` is provided because in the dual path a single nested-dual evaluation produces the value, the first-order sensitivities, and the second-order sensitivities together; obtaining all three through separate calls to `func`, `jac`, and `hess` would repeat the underlying evaluation three times. In the symbolic path `evaluate()` simply combines the three independent C calls and applies the chain rule once.

4.2 Chain rule via dX, dP, dX2, dP2

The seed arguments encode the Jacobian and (optionally) Hessian of an upstream parameter map $\Phi : \theta \mapsto (x, p)$:

- `dX[obs, name, theta]` – $\partial x_i / \partial \theta_a$, indexed per observation row, per state name, per direction θ_a.
- `dP[name, theta]` – $\partial p_j / \partial \theta_a$, constant across observations.
- `dX2[obs, name, theta1, theta2]` – second-order sensitivities $\partial^2 x_i / \partial \theta_a \partial \theta_b$, optional. Defaults to zero (linear / affine Φ).
- `dP2[name, theta1, theta2]` – second-order parameter sensitivities $\partial^2 p_j / \partial \theta_a \partial \theta_b$, optional. Defaults to zero.

Symbols listed in `fixed` have their seed rows zeroed before the contraction; symbols absent from the seeds also contribute zero, not an error.

In the symbolic path the seeds are bundled into one matrix $S \in \mathbb{R}^{n_{\text{obs}} \times n_{\text{diff}} \times n_\theta}$ over the canonical-symbol basis $n_{\text{diff}} = n_x + n_p$. The chain rule for the Jacobian is

$$\left.\frac{\partial y}{\partial \theta}\right|_{\text{obs}} = J(x_{\text{obs}}, p) \cdot S_{\text{obs}}, \tag{22}$$

implemented as one DGEMM per observation in `cppode::chain_jac`. For the Hessian, two contributions combine,

$$\left.\frac{\partial^2 y}{\partial \theta \, \partial \theta^\top}\right|_{\text{obs},\, o} = S_{\text{obs}}^\top \, H_o(x_{\text{obs}}, p) \, S_{\text{obs}} + \sum_i J_{o,i}(x_{\text{obs}}, p) \, S2_{\text{obs},i,\cdot,\cdot}, \tag{23}$$

implemented as two DGEMMs plus one DGEMV per (obs, output) in `cppode::chain_hess`. The second term vanishes when Φ is affine (`dX2` and `dP2` unset).

In the dual path the same contractions are obtained automatically by seeding the dual-number tangent slots. Each input scalar x_i becomes

$$\tilde{x}_i = (x_i + \mathrm{d}X_{i,\cdot}\, \varepsilon_{\text{in}}) + (\mathrm{d}X_{i,\cdot} + \mathrm{d}X2_{i,\cdot,\cdot}\, \varepsilon_{\text{in}})\, \varepsilon_{\text{out}},$$

i.e. a nested dual whose inner tangent slots carry $\partial x_i / \partial \theta$ and whose outer tangent slot k is itself an inner dual carrying $\partial x_i / \partial \theta_k$ as its value and $\partial^2 x_i / \partial \theta_k \partial \theta$ as its inner tangents. The dual-number multiplication rule then propagates both the $S^\top H S$ and the $J \cdot S2$ terms in one nested-arithmetic evaluation; the inner-tangent values of $\tilde{y}$ are read out as $\partial y / \partial \theta$, and the cross slots $\tilde{y}[k][m]$ as $\partial^2 y / \partial \theta_k \partial \theta_m$.

4.3 Pass-through of unmodelled inputs

The `attach.input` flag controls whether unmodelled columns of the input are appended to the output. When `attach.input = TRUE`, columns of the input matrix that do not appear among the named variables of (21) are passed through to the output unchanged: their value rows are copied, their Jacobian rows are zero for variables and the identity for parameters, and their Hessian rows are zero. The convention removes the need to extend g with no-op identities when the observation map does not consume every column of the integrator output, and matches the row-name-driven partial-shape `sens1ini` convention used by `solveODE()`.

References

[1] A. C. Hindmarsh *et al.*, "SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers," *ACM Transactions on Mathematical Software (TOMS)*, vol. 31, no. 3, pp. 363–396, 2005, doi: 10.1145/1089014.1089020.

[2] L. F. Shampine and M. W. Reichelt, "The MATLAB ODE suite," *SIAM Journal on Scientific Computing*, vol. 18, no. 1, pp. 1–22, 1997, doi: 10.1137/S1064827594276424.

[3] E. Hairer and G. Wanner, *Solving ordinary differential equations II: Stiff and differential-algebraic problems.* in Springer series in computational mathematics. Springer Berlin Heidelberg, 2010. Available: https://books.google.de/books?id=zZSv3acps1QC

[4] Ch. Tsitouras, "Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption," *Computers & Mathematics with Applications*, vol. 62, no. 2, pp. 770–775, 2011, doi: 10.1016/j.camwa.2011.06.002.

[5] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving ordinary differential equations I: Nonstiff problems.* in Springer series in computational mathematics. Springer Berlin Heidelberg, 2008. Available: https://books.google.de/books?id=F93u7VcSRyYC

A BDF and NDF coefficients

Source. [2], Table 1; CppODE implementation in `cppode_multistepper.hpp`.

The Numerical Differentiation Formula (NDF) of order q modifies the classical BDFq corrector by adding a term proportional to the Nordsieck predictor error,

$$\sum_{m=1}^{q} \frac{1}{m}\,\nabla^m x_n \;-\; h_n\, f(x_n, t_n) \;-\; \kappa_q\,\gamma_q\,(x_n - x_n^{\mathrm{pred}}) \;=\; 0, \qquad \gamma_q \;=\; \sum_{m=1}^{q} \tfrac{1}{m}.$$

Setting $\kappa_q = 0$ recovers BDFq. The values κ_q adopted by CppODE (selectable at compile time via the argument `useNDF`, default `TRUE`) are tabulated in Table 6, together with the empirical step-size gain over classical BDFq and the resulting stability angle.

q	κ_q	Step-size gain over BDFq	Stability angle
1	-0.1850	$+26\,\%$	A-stable
2	-0.1111	$+26\,\%$	A-stable
3	-0.0823	$+26\,\%$	80°
4	-0.0415	$+12\,\%$	66°
5	0.0000	$0\,\%$	51° (= BDF5)

Table 6: NDF correction coefficients κ_q adopted by CppODE, the empirical step-size gain over classical BDFq at matched local error, and the resulting stability angle.

NDF5 is set to $\kappa_5 = 0$ because the BDF5 stability angle of 51° leaves no headroom for further reduction; NDF5 is therefore identical to BDF5.

B Adams-Moulton coefficients

Source. [1], ℓ-vector formulation as in CVODE; CppODE implementation in `adams_set_coefficients()` inside `cppode_multistepper.hpp`.

In contrast to BDF, the Adams-Moulton coefficients depend on the recent step-size history and are recomputed every step from the Nordsieck array. Define $\xi_j = (t_n - t_{n-j})/h_n$; the ℓ-vector recurrence reads

$$\ell_0(q) \;=\; 1, \qquad \ell_j(q) \;=\; \ell_{j-1}(q) \;+\; \xi_j^{-1}, \qquad j = 1, \dots, q,$$

followed by a polynomial expansion that yields the corrector weights $\beta_0, \dots, \beta_q$ used in the current step. Because the coefficients are step-size adaptive, no static table can reproduce them in general; the canonical fixed-step values for $h_n = h_{n-1} = \cdots$ are given in standard references [5], Table III.1.1 and reduce to the recurrence above in that limit.

The maximum order in CppODE is $q_{\max} = 12$, matching CVODE's default; the order controller may select any $q \in \{1, \dots, 12\}$ at run time.

C Rosenbrock4 coefficients

Source. [3, Sec. IV.7]; identical to the default coefficients in the `rosenbrock4` stepper of Boost.Numeric.Odeint. CppODE implementation in `default_rosenbrock_coefficients` in `cppode_rosenbrock4.hpp`.

A Rosenbrock method of s stages with diagonal coefficient γ is defined, with iteration matrix $W = (\gamma h_n)^{-1} I -$

$J(x_n, t_n)$, by the stage equations

$$W\,k_i \;=\; f\!\left(x_n + \sum_{j<i} \alpha_{ij}\,k_j,\; t_n + c_i\,h_n\right) \;+\; \frac{1}{h_n}\sum_{j<i}\frac{\gamma_{ij}}{\gamma}\,k_j \;+\; h_n\,d_i\,\frac{\partial f}{\partial t},$$

$$x_{n+1} \;=\; x_n + \sum_{i=1}^{s} m_i\,k_i, \qquad \hat{x}_{n+1} \;=\; x_n + \sum_{i=1}^{s} \hat{m}_i\,k_i.$$

The CppODE parameterisation uses $s = 6$ stages with $\gamma = 1/4$. The numerical values of all coefficients are reproduced in the following tables, indexed exactly as in the source code: the diagonal coefficient and the time partials and node positions in Table 7, the stage couplings α_{ij} and γ_{ij} in Tables 8 and 9, the sixth-row weights in Table 10, and the dense-output coefficients in Table 11.

C.1 Diagonal coefficient, time partials, and node positions

γ	0.25	d_1	0.25	c_2	0.386
d_2	-0.1043	d_3	0.1035	c_3	0.21
d_4	0.0362			c_4	0.63

Table 7: Rosenbrock4 diagonal coefficient γ, time-partial coefficients d_i, and stage node positions c_i.

C.2 Stage couplings α_{ij}

	$j=1$	$j=2$	$j=3$	$j=4$
$\alpha_{2,j}$	1.5440000000			
$\alpha_{3,j}$	0.9466785281	0.2557011699		
$\alpha_{4,j}$	3.3148251871	2.8961240160	0.9986419140	
$\alpha_{5,j}$	1.2212245092	6.0191344813	12.5370833293	−0.6878860361

Table 8: Rosenbrock4 stage-coupling coefficients α_{ij}. Row index i is the stage being formed; column index j ranges over earlier stages.

C.3 Stage couplings γ_{ij}

	$j=1$	$j=2$	$j=3$	$j=4$
$\gamma_{2,j}$	−5.6688000000			
$\gamma_{3,j}$	−2.4300933568	−0.2063599157		
$\gamma_{4,j}$	−0.1073529058	−9.5945622510	−20.4702861481	
$\gamma_{5,j}$	7.4964433140	−10.2468043146	−33.9999035282	11.7089089321

Table 9: Rosenbrock4 stage-coupling coefficients γ_{ij}, appearing in the $(1/h_n)\sum_{j<i}(\gamma_{ij}/\gamma)\,k_j$ term of the stage equation. Row and column conventions match Table 8.

C.4 Solution and error weights

The advanced solution and the embedded error estimate are formed from the stage values $k_1, \ldots, k_5$ via

$$x_{n+1} \;=\; x_n + \sum_{i=1}^{5} \alpha_{6,i}\,k_i, \qquad e_{n+1} \;=\; \sum_{i=1}^{5} \gamma_{6,i}\,k_i,$$

with the sixth-row coefficients listed in Table 10.

	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$
$\alpha_{6,i}$	1.2212245092	6.0191344813	12.5370833293	−0.6878860361	1.0
$\gamma_{6,i}$	8.0832467959	−7.9811329881	−31.5215943287	16.3193054312	−6.0588182388

Table 10: Rosenbrock4 sixth-row coefficients. The $\alpha_{6,i}$ values combine the stage values into the advanced solution x_{n+1}; the $\gamma_{6,i}$ values supply the embedded order-3 error estimate e_{n+1}.

C.5 Dense-output coefficients

Continuous output on $[t_n, t_{n+1}]$ is constructed from the five intermediate stage values together with two auxiliary coefficient sets $d_i^{(2)}$ and $d_i^{(3)}$, listed in Table 11, that combine the stages with the step solution to produce an order-3 interpolant.

	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$
$d_i^{(2)}$	10.1262350834	−7.4879958776	−34.8009186156	−7.9927717076	1.0251377233
$d_i^{(3)}$	−0.6762803393	6.0877146517	16.4308432089	24.7672251142	−6.5943891257

Table 11: Rosenbrock4 dense-output coefficients $d_i^{(2)}$ and $d_i^{(3)}$. Together with the stage values they form the order-3 interpolant on $[t_n, t_{n+1}]$ used to serve user output points falling between accepted steps.

D Tsitouras 5(4) coefficients

Source. [4], Table 1 and equations (7)–(8); CppODE implementation in `cppode_tsit5.hpp`.

The Butcher tableau of the Tsitouras 5(4) pair, reproduced in Table 12, has $s = 7$ stages and obeys the simplifying assumption $a_{i,1} = 0$ for $i \geq 2$; the seventh stage is the FSAL stage. Both rows of weights are listed beneath the A matrix: b_i for the order-5 solution and $\hat{b}_i$ for the embedded order-4 estimate.

0							
0.161	0.161						
0.327	−0.0084806555	0.3354806555					
0.9	2.8971530571	−6.3594484900	4.3622954329				
0.9800255409	5.3258648284	−11.7488835641	7.4955393429	−0.0924950664			
1.0	5.8614554429	−12.9209693178	8.1593678986	−0.0715849733	−0.0282690504		
1.0	0.0964607668	0.01	0.4798896504	1.3790085741	−3.2900695154	2.3247105241	
b_i	0.0964607668	0.01	0.4798896504	1.3790085741	−3.2900695154	2.3247105241	0
$\hat{b}_i$	0.0946807557	0.0091835655	0.4877705284	1.2342975669	−2.7077123499	1.8666284182	0.0151515152

Table 12: Butcher tableau of the Tsitouras 5(4) pair. The first column lists the node positions c_i; the centre block is the strictly lower-triangular coefficient matrix a_{ij}. The b_i row gives the order-5 weights, and the $\hat{b}_i$ row gives the embedded order-4 weights.

The seventh stage is $k_7 = f(x_{n+1}, t_n + h_n)$, which is re-used as k_1 at step $n+1$. Because $b_7 = 0$, the order-5 solution does not depend on k_7; because $\hat{b}_7 \neq 0$, the embedded order-4 estimate does. The local-error estimator used by the step controller is

$$e_n \;=\; h_n \sum_{i=1}^{7} (\hat{b}_i - b_i)\, k_i .$$

The seven differences $\hat{b}_i - b_i$ are stored in `cppode_tsit5.hpp` as the coefficients `e1`, ..., `e7` and reproduced in Table 13.

i	$\hat{b}_i - b_i$
1	−0.0017800111
2	−0.0008164345
3	0.0078808780
4	−0.1447110072
5	0.5823571655
6	−0.4580821059
7	0.0151515152

Table 13: Tsit5 embedded error coefficients $\hat{b}_i - b_i$, stored in `cppode_tsit5.hpp` as `e1`, ..., `e7`. The seven values sum to numerical zero, the consistency condition for the embedded pair.

The seven values sum to numerical zero, as required for the consistency of the embedded pair, and the row $a_{7,j}$ of the A matrix coincides with b_j for $j = 1, \ldots, 6$ (the FSAL property). Continuous output on $[t_n, t_{n+1}]$ is delivered by the cubic Hermite interpolant on the accepted-step endpoints (x_n, x_{n+1}) and their derivatives $(f_n, f_{n+1}) = (k_1, k_7)$. Both endpoint derivatives are already in cache from the FSAL property, so no additional right-hand-side evaluations are required for output points.